GENAI BASICS • LESSON 16 • MODULE 6: IDEA TO PRODUCT

Agentic software development

The agent writes the code — you set the task, check it and decide

How a coding agent works, what the tools cost, permission modes, project rules, real incidents and safe practice. Data as of 16.10.2026.



WHAT CHANGED

A chat gives you code to carry over; an agent edits the project and runs the checks itself

CHAT

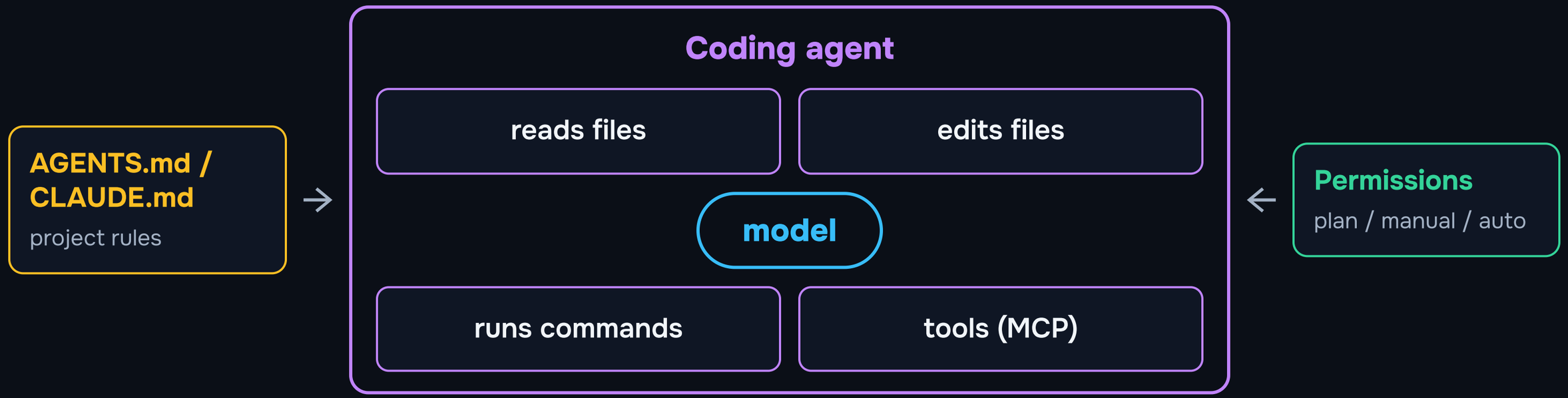
1. **You ask**
   describe what you need
2. **You copy code**
   from the answer into your file
3. **You run it**
   and see what broke
4. **You carry the error back**
   and round you go

CODING AGENT

1. **You set the task**
   goal and “done when…”
2. **The agent edits files**
   reads the project itself
3. **The agent runs tests**
   and fixes the errors
4. **You review**
   read the changes and decide

ANATOMY

A coding agent is a model with hands: it reads, edits, runs and connects tools



The model is the same as in a chat: Claude, GPT, Gemini.

Tools are the agent's hands: files, terminal, MCP connections (L12).

Project rules and permissions are your steering wheel.

TOOLS

You can try for free; daily use is $10–30; heavy use starts at $100

$0

to try it

- **GitHub Copilot Free** · limited agent
- **Codex** · in ChatGPT Free
- **Google Antigravity** · basic limits
- **Cursor** · **Kiro** · **Junie** · free tiers

$10–30

every day

- **Copilot Pro** · $10
- **Claude Code** · in Claude Pro, $20
- **Codex** · in ChatGPT Plus, $20
- **Devin** · **Kiro** · **Cursor** · $20–30

$100+

many agents, big tasks

- **Claude Max** · $100 / $200
- **ChatGPT Pro** · from $100
- **Copilot Max** · $100
- **Cursor** · **Devin** · $100–200

Start free: Codex in ChatGPT, Copilot Free or Antigravity. Prices as of 16.10.2026, per month; Claude Code has no free tier.

THE WORK LOOP

Plan → act → verify → review: the agent turns the loop, you steer



1. **Plan:** the agent reads the project and proposes a plan; no edits yet
2. **Act:** edits on a separate branch — a copy of the project you can throw away
3. **Verify:** tests, build, screenshot
4. **Review:** you read the changes — the diff, “before → after” — and decide

> Anthropic: “explore first, then plan, then code”.

THE TASK

A good agent task says what to do, what not to touch and how to check it is done

VAGUE

```
Add a priority filter.
```

> The agent decides where the filter goes, what to change and when to stop.

AGENT TASK

```
# TASK.md
Goal: filter tickets by priority
      on the /tickets page
Do not touch: database, login
Done when: the filter test
      passes, old tests too
Check: pytest -q
Review: a human reads the diff
```

PROJECT RULES

AGENTS.md is a short project memo the agent reads at the start of every session

```
# AGENTS.md
## Commands
- tests: pytest -q
- run: streamlit run app.py
## Rules
- code in src/, tests in tests/
- do not touch .env or deploy/
- new libraries only
  after asking
```

60,000+

open-source projects already use AGENTS.md, an open format

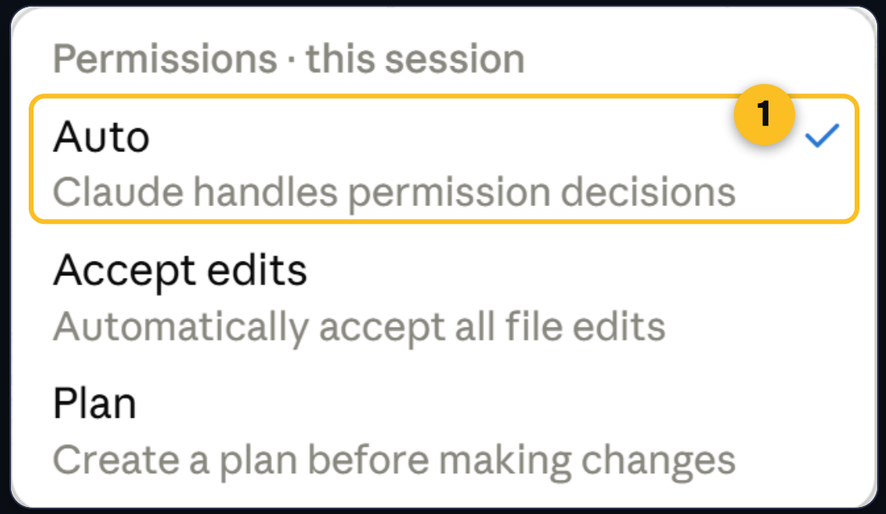
Read by: Codex, Copilot, Cursor, Devin, Junie, Antigravity. Claude Code reads CLAUDE.md, or AGENTS.md if there is none.

Short and concrete: commands and limits, not an essay.

PERMISSIONS

The permission mode decides what the agent does on its own and what needs your OK

CLAUDE.AI/CODE → PERMISSIONS MENU · CAPTURED 16.10.2026



① Auto is the default mode (since October 2026).

93%

of permission prompts people simply approve — they get tired of reading

~17%

of risky actions the auto check still misses (Anthropic's test)

plan
reads and plans only

manual
asks before edits and commands

acceptEdits
edits files and runs simple file commands itself; asks for other commands

auto
a checker model reviews commands · default in the terminal

bypass
no checks — throwaway container only

← safer

faster, riskier →

VERIFY

“If you can’t verify it, don’t ship it”: give the agent a check it can run itself

```
$ pytest -q
..F                                  [100%]
FAILED tests/test_filter.py::
  test_filter_by_priority - TypeError: ...
1 failed, 2 passed in 0.06s
# the agent edits app.py: adds priority
$ pytest -q
...                                  [100%]
3 passed in 0.02s
```

Real output on the homework starter project (the error is shortened).

1. **A test fails:** first write a check for what you need
2. **The agent fixes:** edits the code until the test passes
3. **All tests pass:** the new one and the old ones
4. **You read the diff:** tests don’t see everything — meaning, style, extras

The agent’s undo (Esc Esc, /rewind) reverts file edits, but not terminal commands.

The real undo button is git: each task on its own branch.

SPEED

Experienced developers felt 20% faster with AI — the measurement showed 19% slower

METR, early 2025: 16 experienced developers, 246 tasks in their own projects, tools of that time (Cursor + Claude 3.5/3.7).



> **2026 follow-up:** probably faster now, but the data is weak — people refused to work without AI.

> **Takeaway:** feeling fast ≠ being fast. Measure on your own tasks.

INCIDENTS

An agent does whatever it can reach: keys, databases and other people’s text in the project

9 seconds

PocketOS, 2026: the agent found an access key in an unrelated file and deleted the database with its backups; per the founder, data was later restored

Code freeze

Replit, 2025: the agent deleted SaaStr’s live database although changes were frozen

2,349

secrets leaked: an infected Nx library (a ready-made module developers download) stole keys and made installed AI agents search for secret files, 2025

A key within the agent’s reach = a permission given to the agent.

Clinejection, 2026: a command in a GitHub issue title → the project’s AI bot ran it → publishing keys were stolen → an infected Cline version went out.

BREAK

5 minutes: which small task in your project would you give an agent?

05:00

Next: safe-work rules, practice, homework.

SAFE WORK

Six rules that sharply cut the risk of such incidents

1. **A separate branch per task**
   git is your undo button
2. **No production keys nearby** PocketOS
   not in the code, not in the project's settings files
3. **Sandbox and least privilege** Replit
   a closed environment; deny deleting and pushing code
4. **Other people's text is unverified code** Nx · Clinejection
   READMEs, issues, foreign AGENTS.md, skills, MCP
5. **A human reads every change**
   ideally with fresh eyes or a second agent
6. **Only a human merges and ships**
   Copilot's agent can only propose changes; a human accepts them

The no-checks mode (bypass, --yolo, danger-full-access) — only in a throwaway container or virtual machine.

PRACTICE

Turn a vague request into a task an agent can carry out and check

> “Users want to filter tickets by priority. Please add it.”

1. **TASK.md**
   goal, do not touch, done when, check
2. **AGENTS.md**
   the test command and limits
3. **The agent’s plan**
   plan mode; read what it proposes
4. **Decision**
   accept, reject or refine the task

No agent at hand — do steps 1–2, then swap TASK.md with a neighbour and check their “done when” line. Free to try: Codex in ChatGPT, Copilot Free, Antigravity.

HOMEWORK

Homework: one small task done by an agent and checked by you

1. The homework starter project (app.py + tests) or your own small repository
2. AGENTS.md: commands and limits
3. TASK.md: goal, do not touch, done when, check
4. The agent in plan mode → edits on a separate branch
5. Run the check yourself, save a screenshot, read the diff
6. AGENT_EVIDENCE.md: the agent’s plan, what ran, what changed, your decision

```
genai-homeworks/L16/
├─ AGENTS.md
├─ TASK.md
├─ AGENT_EVIDENCE.md
└─ check.png
```

> Don’t code? Change only the README; the check is a screenshot of the page. No production keys.

LESSON AND COURSE SUMMARY

The agent does the work; the task, the check and the decision stay with us

A task with “done when”

goal, limits and a check command

Plan first

the agent reads and proposes, you approve

Least privilege

branch, sandbox, no production keys

A human decides

reads the diff, merges and ships

Course complete: from your first chat prompt (L02) to an agent that writes code under your supervision (L16). Thank you!